

SIMATS ENGINEERING

DATABASE MANAGEMENT SYSTEMS (CSA0509)

Assessment Tool 1

Code Challenge :

COURSE OUTCOME COVERED

CO4: Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)

Activity Tool : Code Challenge (High)

Assessment Tool Weightage: 35%

QUESTIONS Total Marks: 50

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5

9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	5
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	5

Code of Conduct:

I, **A Mohammad Naheed(192525336)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **A Mohammad Naheed**

Question 1

Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Correct Python Program

```
class HeapFile:
    def __init__(self):
        self.records = []
        self.next_id = 1

    def insert(self, data):
        record = {
            "id": self.next_id,
            "data": data,
            "deleted": False
        }
        self.records.append(record)
        self.next_id += 1
        return record["id"]

    def delete(self, record_id):
        for record in self.records:
            if record["id"] == record_id and not record["deleted"]:
                record["deleted"] = True
                return True
        return False

    def sequential_scan(self):
        return [r for r in self.records if not r["deleted"]]

heap = HeapFile()

id1 = heap.insert("Alice")
```

```

id2 = heap.insert("Bob")
id3 = heap.insert("Charlie")
heap.insert("David")

print("Before deletion:", heap.sequential_scan())
print("Delete Bob:", heap.delete(id2))
print("After deletion:", heap.sequential_scan())

heap.insert("Eve")
print("Final records:", heap.sequential_scan())

```

Expected result: records are inserted in arrival order, the deleted record is ignored during scanning, and new records are appended.

Complexity: Insert $O(1)$ amortized, Delete $O(n)$, Sequential Scan $O(n)$.

Question 2

Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Correct Python Program

```

class Student:
    def __init__(self, student_id, name, department):
        self.student_id = student_id
        self.name = name
        self.department = department

    def __repr__(self):
        return f"{self.student_id}: {self.name} ({self.department})"

class StaticHashTable:
    def __init__(self, size=101):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, student_id):
        return student_id % self.size

    def insert(self, student):
        index = self.hash_function(student.student_id)

        for s in self.table[index]:
            if s.student_id == student.student_id:
                return False

        self.table[index].append(student)
        return True

    def search(self, student_id):
        index = self.hash_function(student_id)

        for student in self.table[index]:
            if student.student_id == student_id:
                return student
        return None

    def delete(self, student_id):
        index = self.hash_function(student_id)

```

```

        for i, student in enumerate(self.table[index]):
            if student.student_id == student_id:
                del self.table[index][i]
                return True
        return False

    def display_bucket(self, index):
        print(f"Bucket {index}: {self.table[index]}")

hash_table = StaticHashTable(101)

# Insert exactly 500 student records
for i in range(1, 501):
    dept = "CSE" if i % 2 == 0 else "ECE"
    hash_table.insert(Student(i, f"Student{i}", dept))

# Demonstrate collision: 101, 202 and 303 all map to bucket 0
hash_table.display_bucket(0)

print("Search 250:", hash_table.search(250))
print("Delete 250:", hash_table.delete(250))
print("Search 250 after deletion:", hash_table.search(250))

```

Hash function: $h(\text{student_id}) = \text{student_id} \bmod 101$. For example, 101, 202 and 303 all map to bucket 0 and are stored in the same chain.

Average complexity with good distribution: Insert/Search/Delete $O(1)$. Worst case: $O(n)$ for a chain.

Question 3

Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Correct Python Program

```

from bisect import bisect_left

class SortedFile:
    def __init__(self):
        self.records = []

    def insert(self, student_id, name, department):
        ids = [record["id"] for record in self.records]
        position = bisect_left(ids, student_id)

        if position < len(self.records) and self.records[position]["id"] == student_id:
            return False

        record = {
            "id": student_id,
            "name": name,
            "department": department
        }
        self.records.insert(position, record)
        return True

    def binary_search(self, student_id):
        low = 0

```

```

        high = len(self.records) - 1

        while low <= high:
            mid = (low + high) // 2
            mid_id = self.records[mid]["id"]

            if mid_id == student_id:
                return self.records[mid]
            elif student_id < mid_id:
                high = mid - 1
            else:
                low = mid + 1

        return None

    def display(self):
        for record in self.records:
            print(record)

file = SortedFile()
file.insert(105, "Rahul", "CSE")
file.insert(101, "Anita", "ECE")
file.insert(110, "Kiran", "IT")
file.insert(103, "Priya", "CSE")
file.insert(107, "Arun", "MECH")

file.display()

result = file.binary_search(107)
print("Search result:", result)

```

Sorted order: 101, 103, 105, 107, 110. Binary search for 107 finds the record efficiently.

Complexity: Binary Search $O(\log n)$; insertion into an array-based sorted file $O(n)$.

Question 4

Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

Correct Python Program

```

class BTreeNode:
    def __init__(self, leaf=False):
        self.keys = []
        self.children = []
        self.leaf = leaf

class BTree:
    def __init__(self, order=4):
        self.order = order
        self.max_keys = order - 1
        self.root = BTreeNode(leaf=True)

    def search(self, key, node=None):
        if node is None:
            node = self.root

```

```

        i = 0
        while i < len(node.keys) and key > node.keys[i]:
            i += 1

        if i < len(node.keys) and key == node.keys[i]:
            return True

        if node.leaf:
            return False

        return self.search(key, node.children[i])

def split_child(self, parent, index):
    child = parent.children[index]
    middle = child.keys[1]

    left = BTreeNode(child.leaf)
    right = BTreeNode(child.leaf)

    left.keys = child.keys[:1]
    right.keys = child.keys[2:]

    if not child.leaf:
        left.children = child.children[:2]
        right.children = child.children[2:]

    parent.keys.insert(index, middle)
    parent.children[index] = left
    parent.children.insert(index + 1, right)

def insert(self, key):
    if self.search(key):
        return False

    if len(self.root.keys) == self.max_keys:
        old_root = self.root
        new_root = BTreeNode(leaf=False)
        new_root.children.append(old_root)
        self.root = new_root
        self.split_child(new_root, 0)

    self.insert_non_full(self.root, key)
    return True

def insert_non_full(self, node, key):
    if node.leaf:
        node.keys.append(key)
        node.keys.sort()
        return

    i = len(node.keys) - 1
    while i >= 0 and key < node.keys[i]:
        i -= 1
    i += 1

    if len(node.children[i].keys) == self.max_keys:
        self.split_child(node, i)
        if key > node.keys[i]:
            i += 1

    self.insert_non_full(node.children[i], key)

```

```

def display(self, node=None, level=0):
    if node is None:
        node = self.root

    print(" " * level + str(node.keys))

    if not node.leaf:
        for child in node.children:
            self.display(child, level + 1)

btree = BTree(order=4)

for key in [10, 20, 5, 6, 12, 30, 7, 17]:
    btree.insert(key)

print("B-Tree:")
btree.display()

for key in [6, 15, 30]:
    print(key, "Found" if btree.search(key) else "Not Found")

```

Correct final structure for this insertion sequence:

```

[10, 20]
[5, 6, 7]
[12, 17]
[30]

```

The original answer's final B-Tree structure is correct for this implementation, but the Python constructors were written incorrectly as `def init`.

Question 5

Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

Correct Python Program

```

class BPlusNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.values = []
        self.next = None

class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(leaf=True)

    def find_leaf(self, key):
        node = self.root

        while not node.leaf:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            node = node.children[i]

```

```

        return node

def search(self, key):
    leaf = self.find_leaf(key)

    for i, k in enumerate(leaf.keys):
        if k == key:
            return leaf.values[i]

    return None

def insert(self, key, value=None):
    if value is None:
        value = key

    leaf = self.find_leaf(key)

    for i, k in enumerate(leaf.keys):
        if k == key:
            leaf.values[i] = value
            return

    i = 0
    while i < len(leaf.keys) and leaf.keys[i] < key:
        i += 1

    leaf.keys.insert(i, key)
    leaf.values.insert(i, value)

    if len(leaf.keys) >= self.order:
        self.split_leaf(leaf)

def split_leaf(self, leaf):
    new_leaf = BPlusNode(leaf=True)
    mid = len(leaf.keys) // 2

    new_leaf.keys = leaf.keys[mid:]
    new_leaf.values = leaf.values[mid:]

    leaf.keys = leaf.keys[:mid]
    leaf.values = leaf.values[:mid]

    new_leaf.next = leaf.next
    leaf.next = new_leaf

    promoted_key = new_leaf.keys[0]

    if leaf is self.root:
        new_root = BPlusNode(leaf=False)
        new_root.keys = [promoted_key]
        new_root.children = [leaf, new_leaf]
        self.root = new_root
    else:
        parent = self.find_parent(self.root, leaf)
        self.insert_into_parent(parent, promoted_key, new_leaf)

def find_parent(self, node, target):
    if node.leaf:
        return None

```



```

        for child in node.children:
            if child is target:
                return node

        for child in node.children:
            parent = self.find_parent(child, target)
            if parent is not None:
                return parent

        return None

def insert_into_parent(self, parent, key, new_child):
    i = 0
    while i < len(parent.keys) and parent.keys[i] < key:
        i += 1

    parent.keys.insert(i, key)
    parent.children.insert(i + 1, new_child)

    if len(parent.keys) >= self.order:
        self.split_internal(parent)

def split_internal(self, node):
    mid = len(node.keys) // 2
    promoted_key = node.keys[mid]

    new_node = BPlusNode(leaf=False)
    new_node.keys = node.keys[mid + 1:]
    new_node.children = node.children[mid + 1:]

    node.keys = node.keys[:mid]
    node.children = node.children[:mid + 1]

    if node is self.root:
        new_root = BPlusNode(leaf=False)
        new_root.keys = [promoted_key]
        new_root.children = [node, new_node]
        self.root = new_root
    else:
        parent = self.find_parent(self.root, node)
        self.insert_into_parent(parent, promoted_key, new_node)

def range_query(self, low, high):
    result = []
    leaf = self.find_leaf(low)

    while leaf is not None:
        for i, key in enumerate(leaf.keys):
            if key > high:
                return result
            if key >= low:
                result.append((key, leaf.values[i]))
        leaf = leaf.next

    return result

def display_leaves(self):
    node = self.root
    while not node.leaf:
        node = node.children[0]

```

```

        while node:
            print(node.keys, end=" -> ")
            node = node.next
        print("None")

tree = BPlusTree(order=4)

for key in [10, 20, 5, 6, 12, 30, 7, 17, 25, 40]:
    tree.insert(key, f"Value-{key}")

print("Leaf-level linked list:")
tree.display_leaves()

print("Search 17:", tree.search(17))
print("Range 10 to 30:", tree.range_query(10, 30))

```

Important correction: `self.values = []` must be initialized in B+ Tree nodes because search and insertion access leaf values. Leaf nodes are linked through `next`, enabling range queries in $O(\log n + k)$.

Question 6

Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Correct Python Program

```

from bisect import bisect_left

class DensePrimaryIndex:
    def __init__(self):
        self.records = []
        self.index = []

    def insert(self, key, name, department):
        keys = [r["id"] for r in self.records]
        position = bisect_left(keys, key)

        if position < len(self.records) and self.records[position]["id"] == key:
            return False

        self.records.insert(position, {
            "id": key,
            "name": name,
            "department": department
        })

        self.build_index()
        return True

    def build_index(self):
        self.index = [
            (record["id"], position)
            for position, record in enumerate(self.records)
        ]

    def search(self, key):
        keys = [entry[0] for entry in self.index]
        position = bisect_left(keys, key)

```

```

        if position < len(keys) and keys[position] == key:
            record_position = self.index[position][1]
            return self.records[record_position]

        return None

    def display_index(self):
        print("Key\tRecord Position")
        for key, position in self.index:
            print(f"{key}\t{position}")

file = DensePrimaryIndex()

for record in [
    (105, "Rahul", "CSE"),
    (101, "Anita", "ECE"),
    (110, "Kiran", "IT"),
    (103, "Priya", "CSE"),
    (107, "Arun", "MECH")
]:
    file.insert(*record)

file.display_index()
print("Search 107:", file.search(107))

```

Dense index entries: (101,0), (103,1), (105,2), (107,3), (110,4). Every record has an index entry.

Search complexity: $O(\log n)$ when binary search is applied to the index keys.

Question 7

Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

Correct Python Program

```

class Bucket:
    def __init__(self, local_depth):
        self.local_depth = local_depth
        self.records = []

class ExtendibleHashing:
    def __init__(self, bucket_capacity=2):
        self.bucket_capacity = bucket_capacity
        self.global_depth = 1
        self.directory = [Bucket(1), Bucket(1)]

    def hash_function(self, key):
        return key

    def get_index(self, key):
        mask = (1 << self.global_depth) - 1
        return self.hash_function(key) & mask

    def insert(self, key):
        while True:
            index = self.get_index(key)
            bucket = self.directory[index]

```

```

        if key in bucket.records:
            return

        if len(bucket.records) < self.bucket_capacity:
            bucket.records.append(key)
            return

        old_depth = bucket.local_depth

        if old_depth == self.global_depth:
            old_directory = self.directory
            self.global_depth += 1
            self.directory = old_directory + old_directory.copy()

        # Split the overflowing bucket
        bucket.local_depth += 1
        new_bucket = Bucket(bucket.local_depth)

        for i in range(len(self.directory)):
            if self.directory[i] is bucket:
                if ((i >> old_depth) & 1) == 1:
                    self.directory[i] = new_bucket

        old_records = bucket.records
        bucket.records = []

        for record in old_records:
            new_index = self.get_index(record)
            self.directory[new_index].records.append(record)

    def display(self):
        seen = {}
        print(f"Global depth = {self.global_depth}")
        print("Index\tBinary\tLocal Depth\tRecords")

        for i, bucket in enumerate(self.directory):
            if id(bucket) not in seen:
                seen[id(bucket)] = len(seen)

            print(
                f"{i}\t{format(i, f'0{self.global_depth}b')}\t\t"
                f"{bucket.local_depth}\t\t{bucket.records}"
            )

ht = ExtendibleHashing(bucket_capacity=2)

# Eight records chosen to make directory growth visible.
for key in [1, 3, 5, 7, 9, 11, 13, 15]:
    print(f"Inserting {key}")
    ht.insert(key)

ht.display()

```

Correction: directory doubling is conditional. It occurs only when the overflowing bucket's local depth equals the global depth. After doubling, the bucket is split and its records are redistributed using the next hash bit.

For the chosen odd-key sequence and capacity 2, repeated overflows can require more than one directory doubling; the exact number depends on the hash function and insertion sequence.

Question 8

Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.

Correct Python Program

```
class SecondaryStorage:
    def __init__(self, num_records, records_per_block):
        self.records_per_block = records_per_block
        self.blocks = []

        for start in range(1, num_records + 1, records_per_block):
            end = min(start + records_per_block, num_records + 1)
            self.blocks.append(list(range(start, end)))

        # Simple in-memory index: record ID -> block number
        self.index = {}

        for block_no, block in enumerate(self.blocks):
            for record_id in block:
                self.index[record_id] = block_no

    def sequential_search(self, record_id):
        io_count = 0

        for block in self.blocks:
            io_count += 1
            if record_id in block:
                return io_count

        return io_count

    def indexed_search(self, record_id):
        if record_id not in self.index:
            return 1 # one index access in this simple simulation

        index_io = 1
        data_io = 1
        return index_io + data_io

storage = SecondaryStorage(100, 10)
record_id = 87

print("Sequential I/O:", storage.sequential_search(record_id))
print("Indexed I/O:", storage.indexed_search(record_id))
```

For 100 records with 10 records per block, there are 10 blocks. Record 87 is in block 8 (zero-based), so sequential retrieval reads blocks 0 through 8 = 9 I/Os. Under the simplified assumption that the index takes one I/O, indexed retrieval takes 1 index I/O + 1 data-block I/O = 2 I/Os.

Question 9

Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

Correct Python Program

```
import bisect
import random
import time

class SortedFile:
    def __init__(self, num_records, records_per_block):
        self.records_per_block = records_per_block
        self.records = [(i, f"Student-{i}") for i in range(num_records)]

class DenseIndex:
    def __init__(self, sorted_file):
        self.file = sorted_file
        self.index = [
            (record[0], position)
            for position, record in enumerate(self.file.records)
        ]
        self.keys = [key for key, _ in self.index]

    def search(self, key):
        pos = bisect.bisect_left(self.keys, key)

        if pos < len(self.keys) and self.keys[pos] == key:
            record_position = self.index[pos][1]
            return self.file.records[record_position]

        return None

class SparseIndex:
    def __init__(self, sorted_file):
        self.file = sorted_file
        self.index = []

        for position in range(
            0, len(self.file.records), self.file.records_per_block
        ):
            key = self.file.records[position][0]
            self.index.append((key, position))

        self.keys = [key for key, _ in self.index]

    def search(self, key):
        block_index = bisect.bisect_right(self.keys, key) - 1

        if block_index < 0:
            return None

        start = self.index[block_index][1]
        end = min(
            start + self.file.records_per_block,
            len(self.file.records)
        )

        for position in range(start, end):
            record = self.file.records[position]

            if record[0] == key:
```

```

        return record

        if record[0] > key:
            break

    return None

def measure_time(index, keys, repetitions=5):
    total = 0.0

    for _ in range(repetitions):
        start = time.perf_counter()

        for key in keys:
            index.search(key)

        total += time.perf_counter() - start

    return total / repetitions

NUM_RECORDS = 100_000
RECORDS_PER_BLOCK = 100

data_file = SortedFile(NUM_RECORDS, RECORDS_PER_BLOCK)
dense = DenseIndex(data_file)
sparse = SparseIndex(data_file)

test_keys = [random.randint(0, NUM_RECORDS - 1) for _ in range(10_000)]

print("Dense result:", dense.search(54321))
print("Sparse result:", sparse.search(54321))

dense_time = measure_time(dense, test_keys)
sparse_time = measure_time(sparse, test_keys)

print(f"Dense index time : {dense_time:.6f} seconds")
print(f"Sparse index time: {sparse_time:.6f} seconds")

print("Dense index entries :", len(dense.index))
print("Sparse index entries:", len(sparse.index))

```

For 100,000 records and 100 records per block, the dense index has 100,000 entries while the sparse index has 1,000 entries. The exact timing must be produced by running the program on the actual computer; a made-up 'typical' timing should not be presented as measured output.

Dense index: $O(N)$ index space and $O(\log N)$ binary-search lookup. Sparse index: $O(N/B)$ index space, $O(\log(N/B))$ index search plus a scan within the selected block.

Question 10

Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Correct Python Program

```
class BPlusNode:
    def __init__(self, leaf=False):
        self.leaf = leaf
        self.keys = []
        self.children = []
        self.values = []
        self.next = None

class BPlusTree:
    def __init__(self, order=4):
        self.order = order
        self.root = BPlusNode(leaf=True)

    def find_leaf(self, key):
        node = self.root

        while not node.leaf:
            i = 0
            while i < len(node.keys) and key >= node.keys[i]:
                i += 1
            node = node.children[i]

        return node

    def search(self, key):
        leaf = self.find_leaf(key)

        for i, k in enumerate(leaf.keys):
            if k == key:
                return leaf.values[i]

        return None

    def insert(self, key, value):
        leaf = self.find_leaf(key)

        for i, k in enumerate(leaf.keys):
            if k == key:
                leaf.values[i] = value
                return

        i = 0
        while i < len(leaf.keys) and leaf.keys[i] < key:
            i += 1
```



```

leaf.keys.insert(i, key)
leaf.values.insert(i, value)

if len(leaf.keys) >= self.order:
    self.split_leaf(leaf)

def split_leaf(self, leaf):
    new_leaf = BPlusNode(leaf=True)
    mid = len(leaf.keys) // 2

    new_leaf.keys = leaf.keys[mid:]
    new_leaf.values = leaf.values[mid:]

    leaf.keys = leaf.keys[:mid]
    leaf.values = leaf.values[:mid]

    new_leaf.next = leaf.next
    leaf.next = new_leaf

    promoted_key = new_leaf.keys[0]

    if leaf is self.root:
        new_root = BPlusNode(leaf=False)
        new_root.keys = [promoted_key]
        new_root.children = [leaf, new_leaf]
        self.root = new_root
    else:
        parent = self.find_parent(self.root, leaf)
        self.insert_into_parent(parent, promoted_key, new_leaf)

def find_parent(self, node, target):
    if node.leaf:
        return None

    for child in node.children:
        if child is target:
            return node

    for child in node.children:
        result = self.find_parent(child, target)
        if result is not None:
            return result

    return None

def insert_into_parent(self, parent, key, new_child):
    i = 0
    while i < len(parent.keys) and parent.keys[i] < key:
        i += 1

    parent.keys.insert(i, key)
    parent.children.insert(i + 1, new_child)

    if len(parent.keys) >= self.order:
        self.split_internal(parent)

def split_internal(self, node):
    mid = len(node.keys) // 2
    promoted_key = node.keys[mid]

    new_node = BPlusNode(leaf=False)

```

```

new_node.keys = node.keys[mid + 1:]
new_node.children = node.children[mid + 1:]

node.keys = node.keys[:mid]
node.children = node.children[:mid + 1]

if node is self.root:
    new_root = BPlusNode(leaf=False)
    new_root.keys = [promoted_key]
    new_root.children = [node, new_node]
    self.root = new_root
else:
    parent = self.find_parent(self.root, node)
    self.insert_into_parent(parent, promoted_key, new_node)

def traverse(self, node=None, level=0):
    if node is None:
        node = self.root

    kind = "Leaf" if node.leaf else "Internal"
    print(" " * level + f"{kind}: {node.keys}")

    if not node.leaf:
        for child in node.children:
            self.traverse(child, level + 1)

def traverse_leaves(self):
    node = self.root

    while not node.leaf:
        node = node.children[0]

    print("Leaf-level traversal:")
    while node:
        print(node.keys, end=" -> ")
        node = node.next
    print("None")

def range_query(self, low, high):
    result = []
    node = self.find_leaf(low)

    while node:
        for i, key in enumerate(node.keys):
            if key > high:
                return result

            if low <= key <= high:
                result.append((key, node.values[i]))

        node = node.next

    return result

tree = BPlusTree(order=4)

records = [
    (10, "Alice"),
    (5, "Bob"),
    (20, "Charlie"),

```

```
(15, "David"),
(30, "Eva"),
(25, "Frank"),
(35, "Grace"),
(40, "Henry"),
(45, "Ivy"),
(50, "Jack"),
(12, "Kiran"),
(18, "Latha"),
(28, "Manoj"),
(33, "Nisha"),
(42, "Omar"),
(48, "Priya")
]

for key, name in records:
    tree.insert(key, name)

print("B+ Tree Structure")
tree.traverse()

print("\nLeaf-level linked list")
tree.traverse_leaves()

print("\nRange Query: 15 to 45")
result = tree.range_query(15, 45)

for key, value in result:
    print(f"Key = {key}, Record = {value}")
```